DYNAMIC PROGRAMMING

DYNAMIC PROGRAMMING

Concept

- To solve some problems, we often make decisions or calculations based on subproblems, which are often redundant
- The core idea is to store the results of solved subproblems and reuse them later instead of recalculating

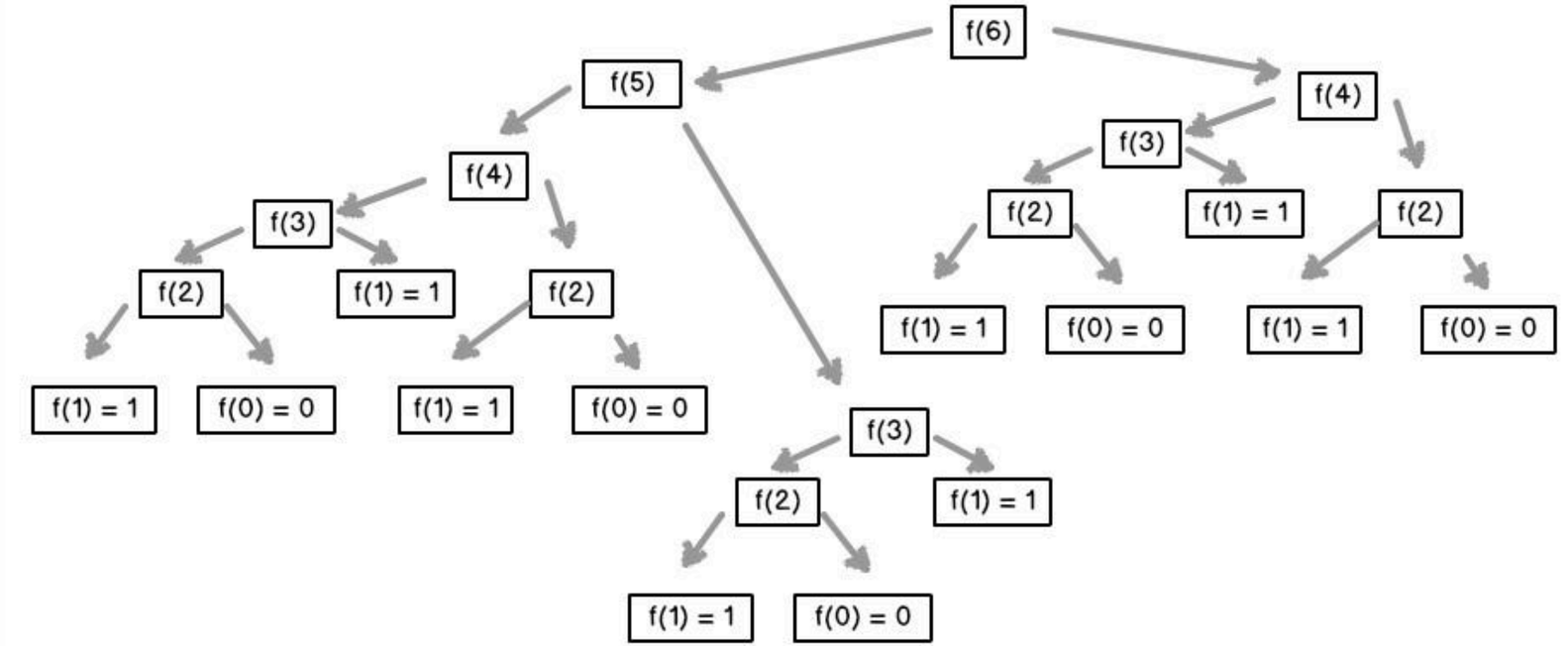
When this will work?

- Overlapping Subproblems — the same subproblems are solved multiple times in a naive approachs
- Optimal Substructure — the solution to the main problem depends on the solutions to its subproblems

N-TH FIBONACCI NUMBER

Most original one, we define fibonacci function as

$$fib(n) = \begin{cases} 0 & ; n=0 \\ 1 & ; n=1 \\ fib(n-1)+fib(n-2) & ; otherwise \end{cases}$$



N-TH FIBONACCI NUMBER

Most original one, we define fibonacci function as

$$fib(n) = \begin{cases} 0 & ; n=0 \\ 1 & ; n=1 \\ fib(n-1)+fib(n-2) & ; otherwise \end{cases}$$

what we know so far

↓

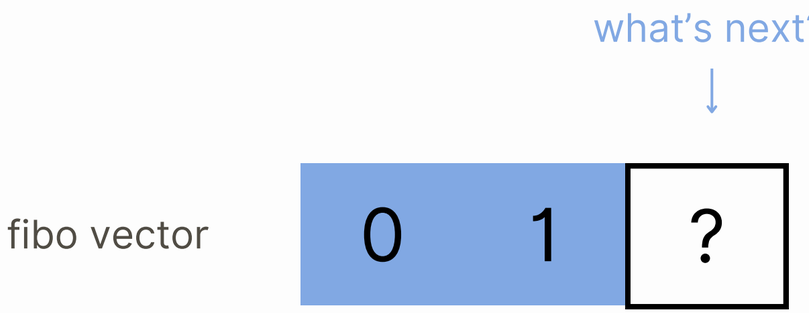
fibo vector



N-TH FIBONACCI NUMBER

Most original one, we define fibonacci function as

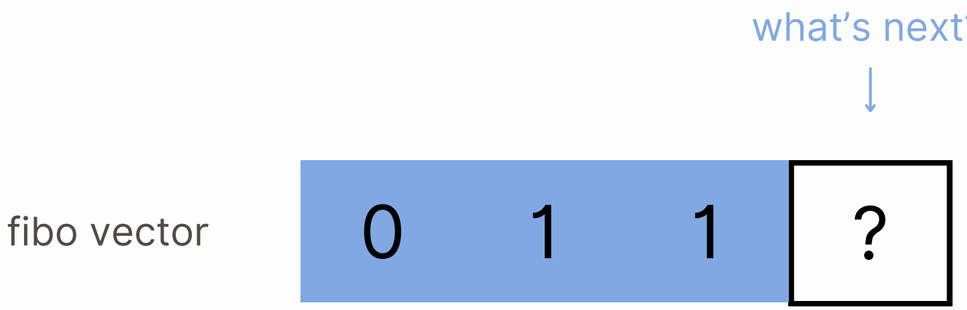
$$fib(n) = \begin{cases} 0 & ; n=0 \\ 1 & ; n=1 \\ fib(n-1)+fib(n-2) & ; otherwise \end{cases}$$



N-TH FIBONACCI NUMBER

Most original one, we define fibonacci function as

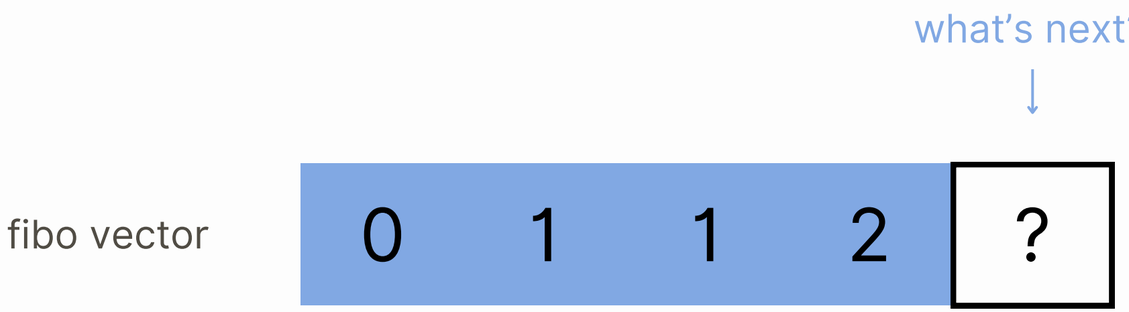
$$fib(n) = \begin{cases} 0 & ; n=0 \\ 1 & ; n=1 \\ fib(n-1)+fib(n-2) & ; otherwise \end{cases}$$



N-TH FIBONACCI NUMBER

Most original one, we define fibonacci function as

$$fib(n) = \begin{cases} 0 & ; n=0 \\ 1 & ; n=1 \\ fib(n-1)+fib(n-2) & ; otherwise \end{cases}$$



N-TH FIBONACCI NUMBER

Most original one, we define fibonacci function as

$$fib(n) = \begin{cases} 0 & ; n=0 \\ 1 & ; n=1 \\ fib(n-1)+fib(n-2) & ; otherwise \end{cases}$$

fibo vector



RANGE SUM

Task: Given array **A** of **n** integers and **q** queries, each query ask for sum in range **[l,r]**

Approach: each query we calculate sum in range [l,r]



q1 : [2,7] → 1+2+7+3+1+8 = 22
q2: [4,9] → 7+3+1+8+3+6 = 28
q3: [1,5] → 5+1+2+7+3 = 17

Time complexity: O(qn)

RANGE SUM

Task: Given array **A** of **n** integers and **q** queries, each query ask for sum in range **[l,r]**

A



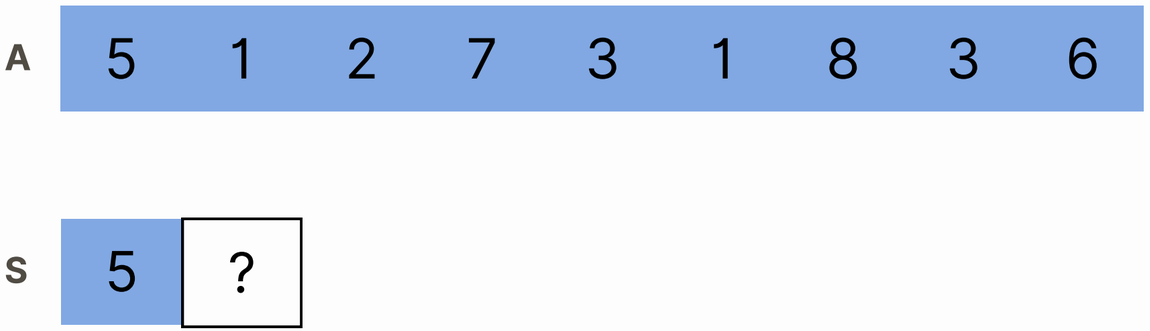
Approach: each query we calculate sum in range [l,r]

q1 : [2,7] → 1+2+7+3+1+8 = 22
q2: [4,9] → 7+3+1+8+3+6 = 28
q3: [1,5] → 5+1+2+7+3 = 18

Time complexity: O(qn)

RANGE SUM

Task: Given array **A** of **n** integers and **q** queries, each query ask for sum in range **[l,r]**

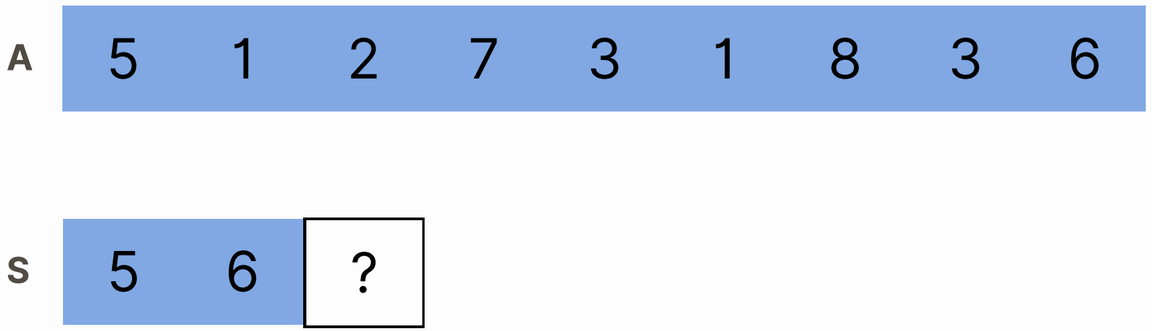


Approach:

- we calculate summation from 1 to every position in advance

RANGE SUM

Task: Given array **A** of **n** integers and **q** queries, each query ask for sum in range **[l,r]**

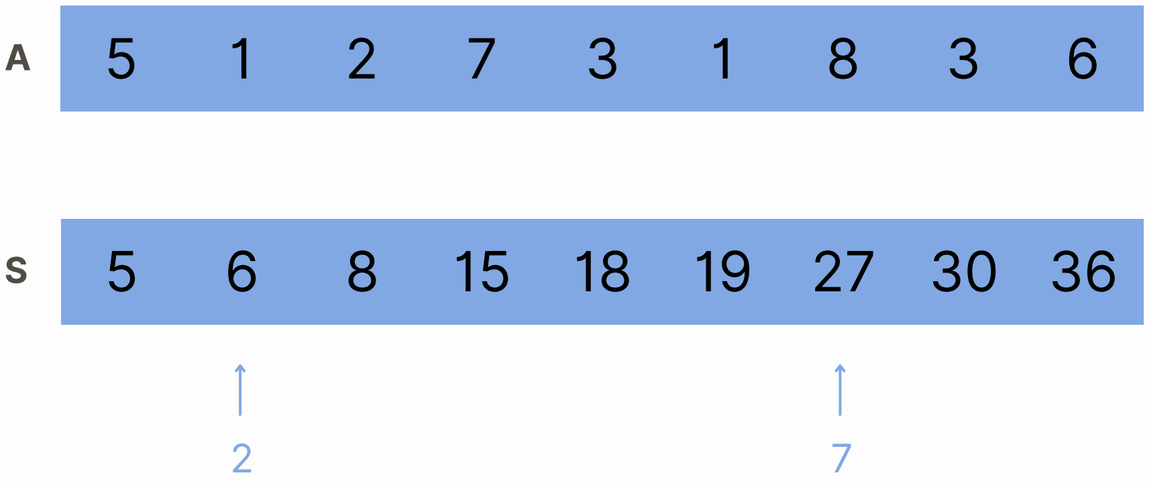


Approach:

- we calculate summation from 1 to every position in advance

RANGE SUM

Task: Given array **A** of **n** integers and **q** queries, each query ask for sum in range **[l,r]**



Approach:

- we calculate summation from 1 to every position in advance
- each query range [l,r] calculate **sum[1,r]-sum[1,l-1](if exist)**

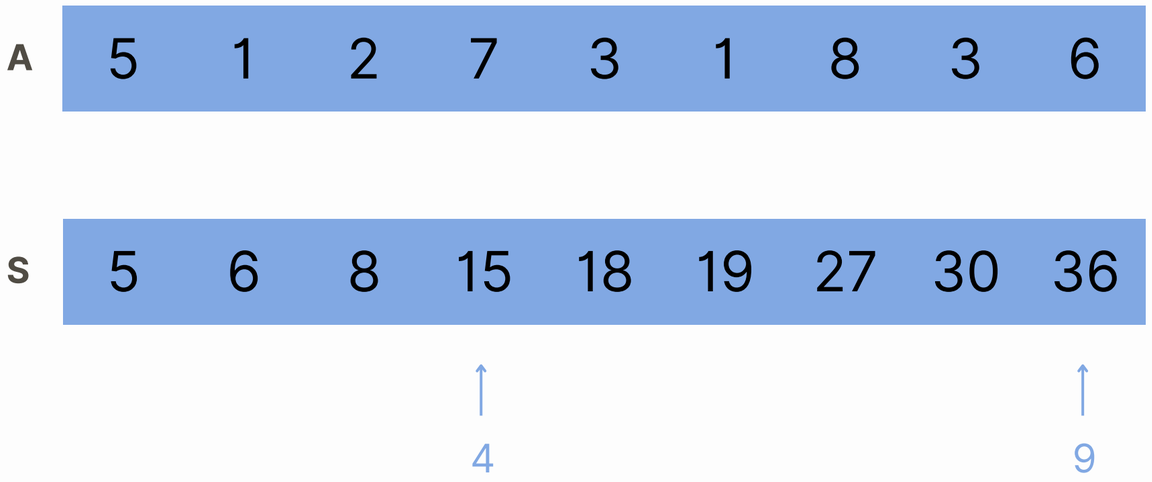
q1 : [2,7] → S[7] - S[2-1] = 27-5 = 21

q2:

q3:

RANGE SUM

Task: Given array **A** of **n** integers and **q** queries, each query ask for sum in range **[l,r]**



Approach:

- we calculate summation from 1 to every position in advance
- each query range [l,r] calculate **sum[1,r]-sum[1,l-1](if exist)**

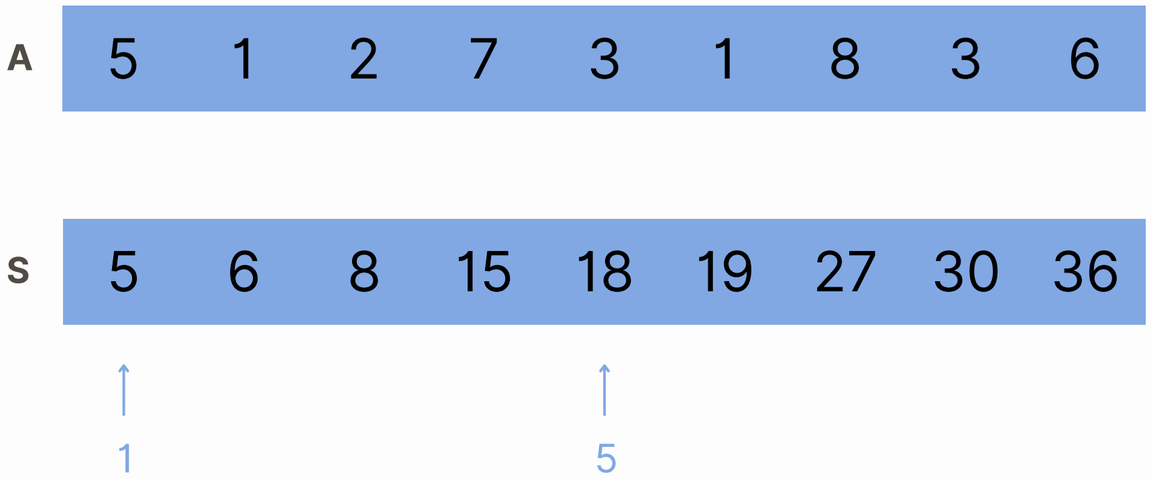
q1 : [2,7] → S[7] - S[2-1] = 27-5 = 21

q2: [4,9] → S[9] - S[4-1] = 36-8 = 28

q3:

RANGE SUM

Task: Given array **A** of **n** integers and **q** queries, each query ask for sum in range **[l,r]**



Approach:

- we calculate summation from 1 to every position in advance
- each query range [l,r] calculate **sum[1,r]-sum[1,l-1](if exist)**

q1 : [2,7] → S[7] - S[2-1] = 27-5 = 21

q2: [4,9] → S[9] - S[4-1] = 36-8 = 28

q3: [1,5] → S[5] = 18

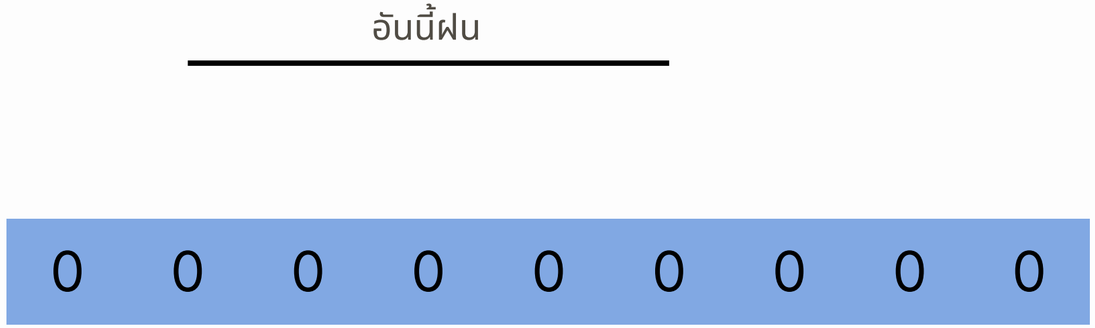
Time complexity: O(n) + O(q)

APPLYING OF PREFIX SUM

It's the rainy season, and Mr. A has placed **n** empty buckets in a straight line, numbered from 1 to n.

Over the course of **q** nights, rain falls on specific ranges of buckets. Each night, the rain covers buckets from position **l to r** (inclusive), and each bucket in that range gains 1 unit of water.

Determine the final water level in each bucket.



APPLYING OF PREFIX SUM

It's the rainy season, and Mr. A has placed **n** empty buckets in a straight line, numbered from 1 to n.

Over the course of **q** nights, rain falls on specific ranges of buckets. Each night, the rain covers buckets from position **l to r** (inclusive), and each bucket in that range gains 1 unit of water.

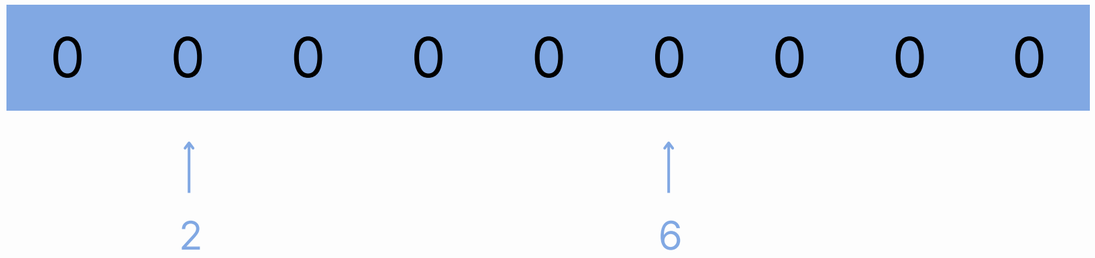
Determine the final water level in each bucket.



APPLYING OF PREFIX SUM

Approach:

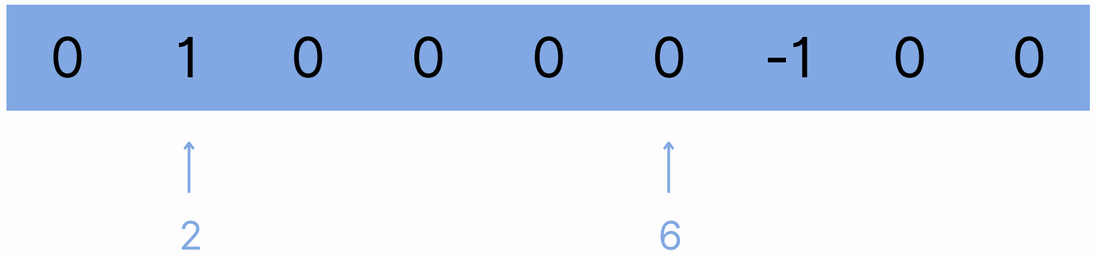
- If rain cover l-th to r-th bucket
  - Add 1 to the l-th bucket
  - Minus 1 to the (r+1)-th bucket



APPLYING OF PREFIX SUM

Approach:

- If rain cover l-th to r-th bucket
  - Add 1 to the l-th bucket
  - Minus 1 to the (r+1)-th bucket
- Repeat for all raining event



APPLYING OF PREFIX SUM

Approach:

- If rain cover l-th to r-th bucket
  - Add 1 to the l-th bucket
  - Minus 1 to the (r+1)-th bucket
- Repeat for all raining event

[2,6], [4,8] and [1,3]

[2,6]

0	1	0	0	0	0	-1	0	0

[4,8]

0	1	0	1	0	0	-1	0	-1

[1,3]

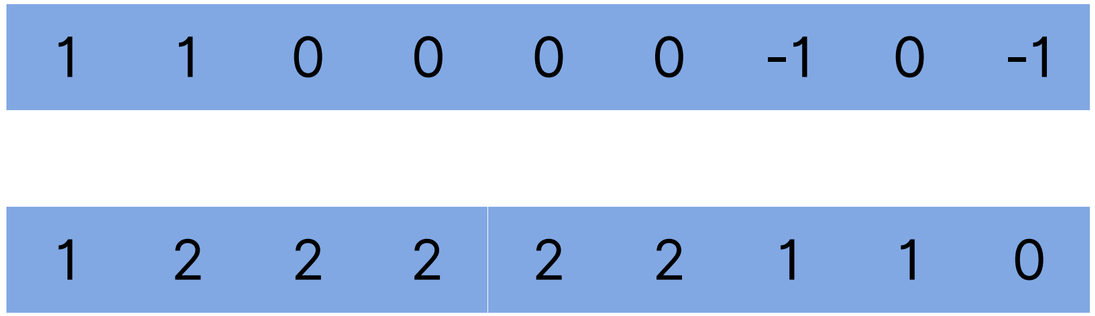
1	1	0	0	0	0	-1	0	-1

APPLYING OF PREFIX SUM

Approach:

- If rain cover l-th to r-th bucket
  - Add 1 to the l-th bucket
  - Minus 1 to the (r+1)-th bucket
- Repeat for all raining event
- Perform Prefix Sum to final array, final answer

[2,6], [4,8] and [1,3]



APPLYING OF PREFIX SUM

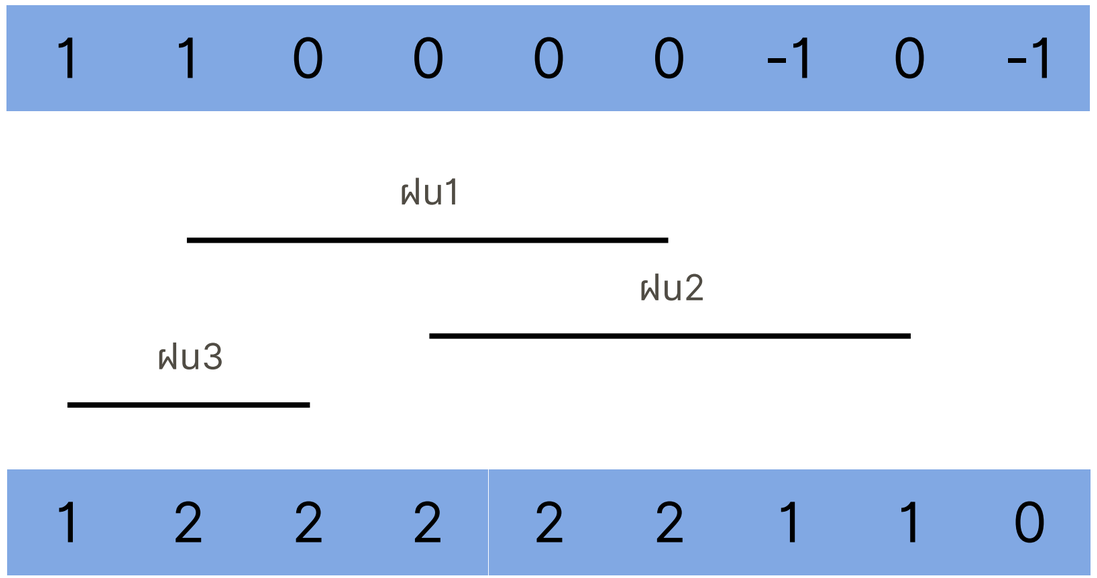
Approach:

- If rain cover l-th to r-th bucket
  - Add 1 to the l-th bucket
  - Minus 1 to the (r+1)-th bucket
- Repeat for all raining event
- Perform Prefix Sum to final array, final answer

Time Complexity: O(q) + O(n)

Naive TC: O(qn)

[2,6], [4,8] and [1,3]



SUBSET SUM

Given a set of n integers and a target sum k,

can you choose a subset of these numbers

such that their sum is exactly k?



SUBSET SUM

Given a set of n integers and a target sum k,

can you choose a subset of these numbers

such that their sum is exactly k?



What we know so far?

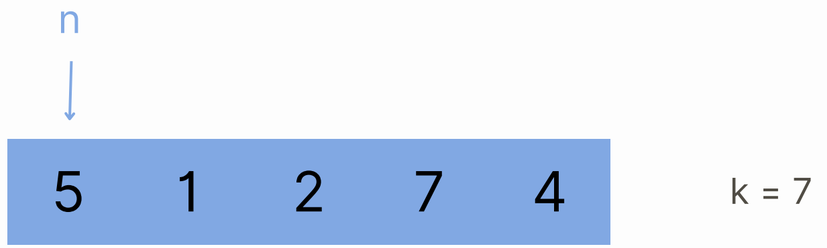
- if we have nothing → {0} **(base case)**
- have only first element → {0} U {0+5} → {0,5}
- first 2 → {0,5} U {0+1, 5+1} → {0,1,5,6}
- and so on

SUBSET SUM

Given a set of n integers and a target sum k,

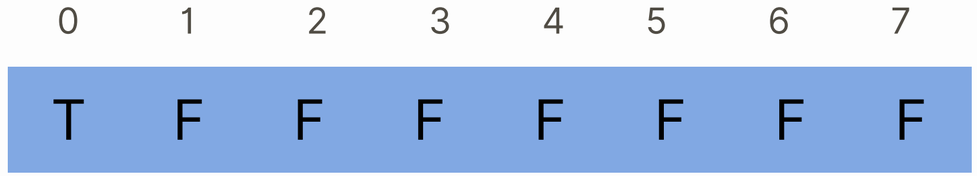
can you choose a subset of these numbers

such that their sum is exactly k?



Approach:

- Build boolean array **dp** of size **k+1**, where dp[i] = T means that a subset exists with target sum i
- base case is we can always achieve target sum 0. So **dp[0] = T** and the rest is **F**
- for each integer **n**, we know all position i that dp[i]=T so far, mark all dp[i+n] = T (if exist)

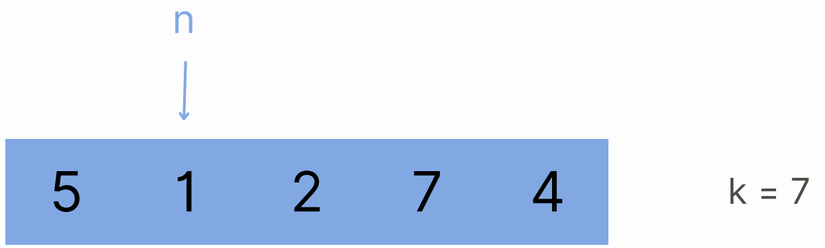


SUBSET SUM

Given a set of n integers and a target sum k,

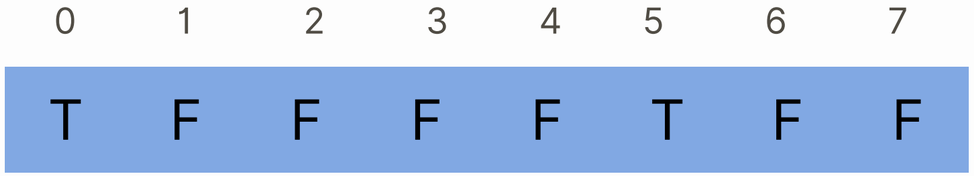
can you choose a subset of these numbers

such that their sum is exactly k?



Approach:

- Build boolean array **dp** of size **k+1**, where dp[i] = T means that a subset exists with target sum i
- base case is we can always achieve target sum 0. So **dp[0] = T** and the rest is **F**
- for each integer **n**, we know all position i that dp[i]=T so far, mark all dp[i+n] = T (if exist)

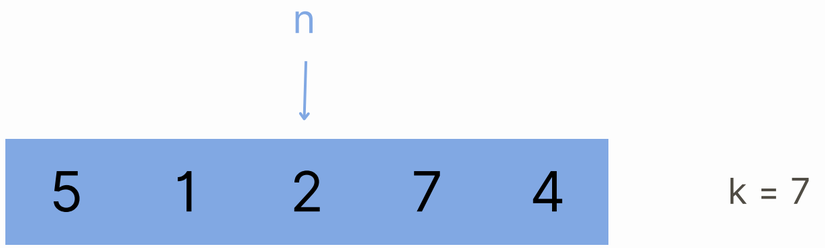


SUBSET SUM

Given a set of n integers and a target sum k,

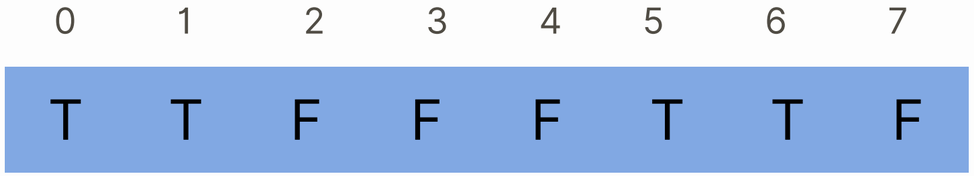
can you choose a subset of these numbers

such that their sum is exactly k?



Approach:

- Build boolean array **dp** of size **k+1**, where dp[i] = T means that a subset exists with target sum i
- base case is we can always achieve target sum 0. So **dp[0] = T** and the rest is **F**
- for each integer **n**, we know all position i that dp[i]=T so far, mark all dp[i+n] = T (if exist)

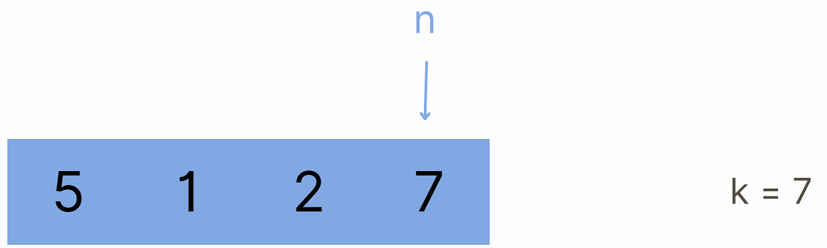


SUBSET SUM

Given a set of n integers and a target sum k,

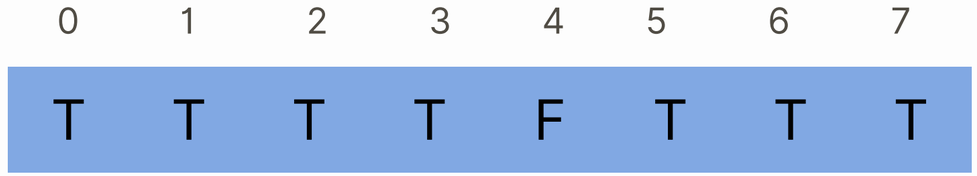
can you choose a subset of these numbers

such that their sum is exactly k?



Approach:

- Build boolean array **dp** of size **k+1**, where dp[i] = T means that a subset exists with target sum i
- base case is we can always achieve target sum 0. So **dp[0] = T** and the rest is **F**
- for each integer **n**, we know all position i that dp[i]=T so far, mark all dp[i+n] = T (if exist)



SUBSET SUM

Given a set of n integers and a target sum k,

can you choose a subset of these numbers

such that their sum is exactly k?



k = 7

Approach:

- Build boolean array **dp** of size **k+1**, where dp[i] = T means that a subset exists with target sum i
- base case is we can always achieve target sum 0. So **dp[0] = T** and the rest is **F**
- for each integer **n**, we know all position i that dp[i]=T so far, mark all dp[i+n] = T (if exist)



Time Complexity: O(kn)

SUBSET SUM

Given a set of n integers and a target sum k,

can you choose a subset of these numbers

such that their sum is exactly k?



k = 7

Approach:

- Build boolean array **dp** of size **k+1**, where dp[i] = T means that a subset exists with target sum i
- base case is we can always achieve target sum 0. So **dp[0] = T** and the rest is **F**
- for each integer **n**, we know all position i that dp[i]=T so far, mark all dp[i+n] = T (if exist)



SUBSET SUM

```
vector<int> a = {5,1,2,7};
 int k = 7;
 vector<bool> dp(k+1, false);
 dp[0] = true;
 for(int i=0;i<a.size();i++)
 {
  for(int j=k-a[i];j>=0;j--)
  {
    dp[j+a[i]] = dp[j+a[i]] || dp[j];
  }
 }
 cout << dp.back();
```

Here, instead of looping forward from 0 to k-a[i], we loop backward from k-a[i] because looping forward will cause redundant number using

Example: if a[i] = 2, and we looping forward

we found dp[0] = T, we mark dp[2] = T

after this we will found dp[2] = T, then mark dp[4] = T

after this we will found dp[4] = T (again), and mark dp[6] = T

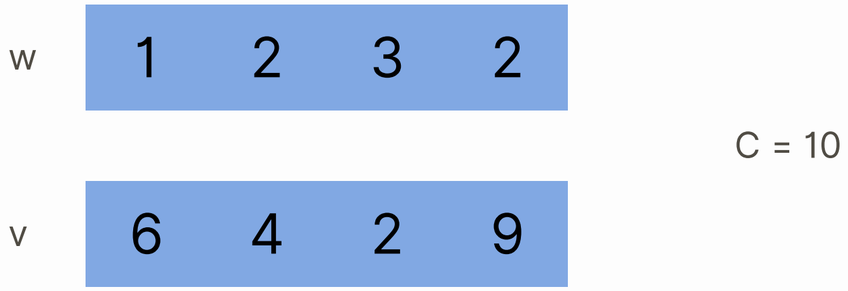
0/1 KNAPSACK

Given n items, each with a weight w[i] and a value v[i],

and a maximum capacity C of a knapsack.

Choose a subset of items (each at most once) such that:

- The total weight does not exceed C, and
- The total value is maximized



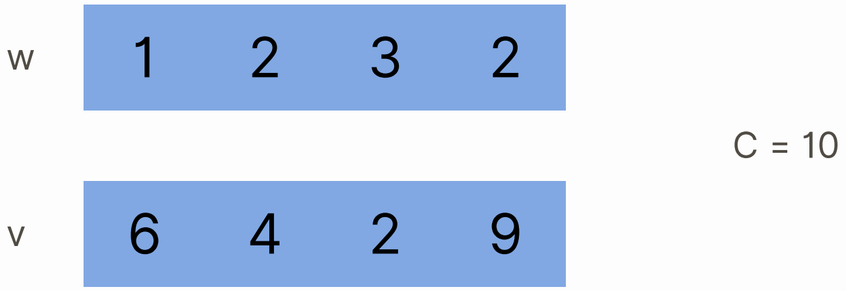
0/1 KNAPSACK

Given n items, each with a weight w[i] and a value v[i],

and a maximum capacity C of a knapsack.

Choose a subset of items (each at most once) such that:

- The total weight does not exceed C, and
- The total value is maximized



What we know so far?

- if we have nothing → (0→0)
- have only first item → (0→0), (1→6)
- first 2 → (0→0), (1→6), (2→4), (3→10)
- and so on

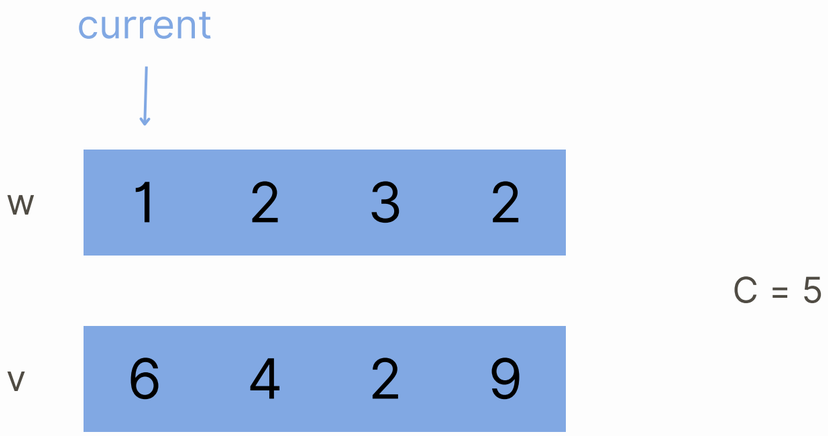
0/1 KNAPSACK

Given n items, each with a weight w[i] and a value v[i],

and a maximum capacity C of a knapsack.

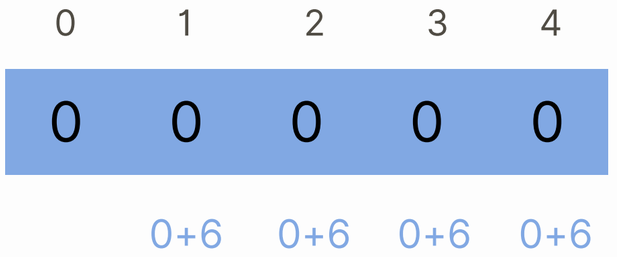
Choose a subset of items (each at most once) such that:

- The total weight does not exceed C, and
- The total value is maximized



Approach:

- Build int array **dp** of size **C+1**, where dp[i] means maximum value can achieve with total weight <= i
- Base case is we pick nothing, all dp[i] = 0
- for each item (w,v). With each total weight i, if we can add this item without breaking the bag, so we can achieve value dp[i]+v at weight i+w, compare with old value at weight i+w
  - dp[i+w] = max(dp[i+w], dp[i]+v)



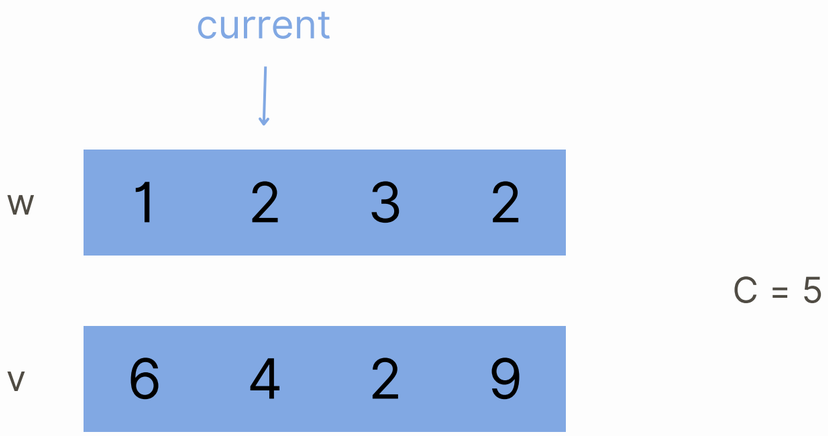
0/1 KNAPSACK

Given n items, each with a weight w[i] and a value v[i],

and a maximum capacity C of a knapsack.

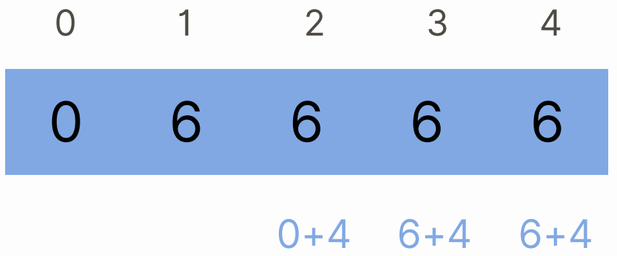
Choose a subset of items (each at most once) such that:

- The total weight does not exceed C, and
- The total value is maximized



Approach:

- Build int array **dp** of size **C+1**, where dp[i] means maximum value can achieve with total weight <= i
- Base case is we pick nothing, all dp[i] = 0
- for each item (w,v). With each total weight i, if we can add this item without breaking the bag, so we can achieve value dp[i]+v at weight i+w, compare with old value at weight i+w
  - dp[i+w] = max(dp[i+w], dp[i]+v)



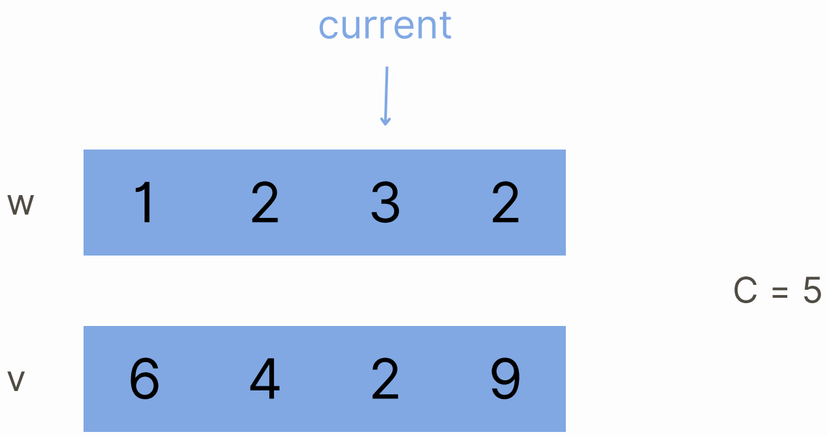
0/1 KNAPSACK

Given n items, each with a weight w[i] and a value v[i],

and a maximum capacity C of a knapsack.

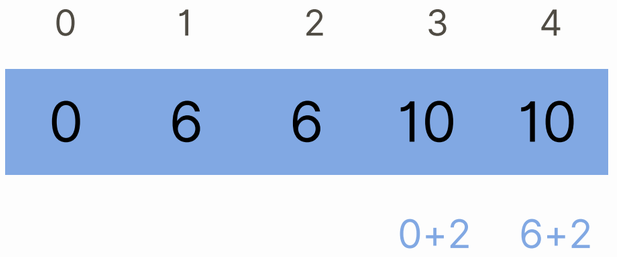
Choose a subset of items (each at most once) such that:

- The total weight does not exceed C, and
- The total value is maximized



Approach:

- Build int array **dp** of size **C+1**, where dp[i] means maximum value can achieve with total weight <= i
- Base case is we pick nothing, all dp[i] = 0
- for each item (w,v). With each total weight i, if we can add this item without breaking the bag, so we can achieve value dp[i]+v at weight i+w, compare with old value at weight i+w
  - dp[i+w] = max(dp[i+w], dp[i]+v)



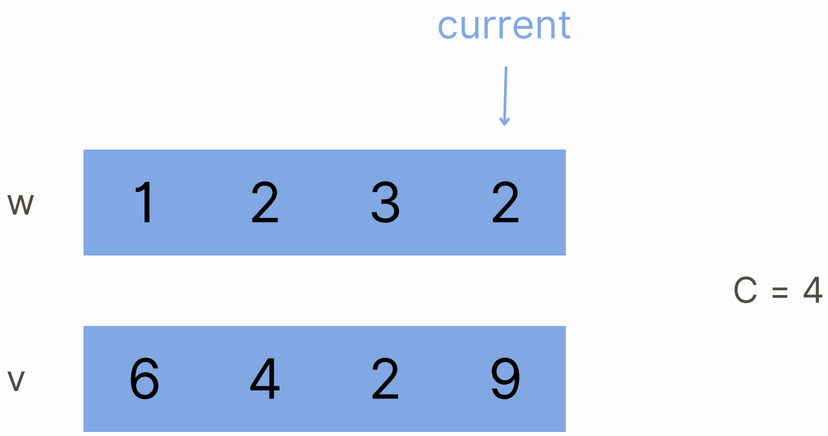
0/1 KNAPSACK

Given n items, each with a weight w[i] and a value v[i],

and a maximum capacity C of a knapsack.

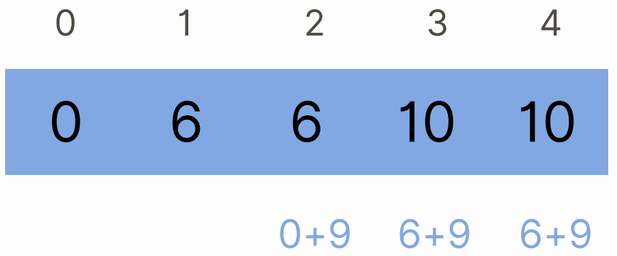
Choose a subset of items (each at most once) such that:

- The total weight does not exceed C, and
- The total value is maximized



Approach:

- Build int array **dp** of size **C+1**, where dp[i] means maximum value can achieve with total weight <= i
- Base case is we pick nothing, all dp[i] = 0
- for each item (w,v). With each total weight i, if we can add this item without breaking the bag, so we can achieve value dp[i]+v at weight i+w, compare with old value at weight i+w
  - dp[i+w] = max(dp[i+w], dp[i]+v)



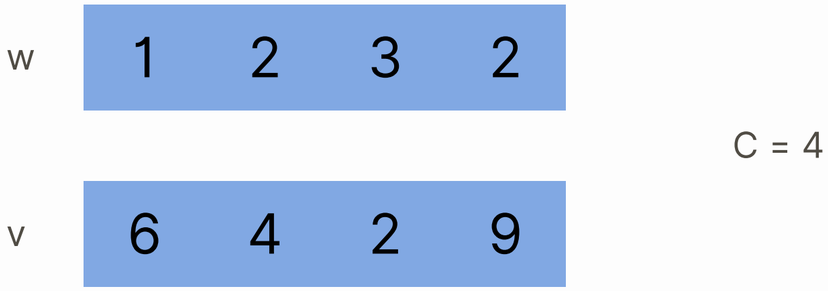
0/1 KNAPSACK

Given n items, each with a weight w[i] and a value v[i],

and a maximum capacity C of a knapsack.

Choose a subset of items (each at most once) such that:

- The total weight does not exceed C, and
- The total value is maximized



Approach:

- Build int array **dp** of size **C+1**, where dp[i] means maximum value can achieve with total weight <= i
- Base case is we pick nothing, all dp[i] = 0
- for each item (w,v). With each total weight i, if we can add this item without breaking the bag, so we can achieve value dp[i]+v at weight i+w, compare with old value at weight i+w
  - dp[i+w] = max(dp[i+w], dp[i]+v)



Time Complexity: O(Cn)

0/1 KNAPSACK

```
vector<int> w = {1,2,3,2};
 vector<int> v = {6,4,2,9};
 int c = 4;
 vector<int> dp(c+1, 0);
 dp[0] = 0;
 for(int i=0;i<w.size();i++)
 {
  for(int j=c;j>=w[i];j--)
  {
   dp[j] = max(dp[j], dp[j-w[i]] + v[i]);
  }
 }
 cout << dp.back();
```

MAKING PALINDROME

Given string s, you can perform insertion or deletion at any position to make s plaindrome. What is the minimum operations needed

Example : **“abab”** → insert “a” at the back so string is **“ababa”**

S = “abcea”

MAKING PALINDROME

S = “abcea”

What we know so far?

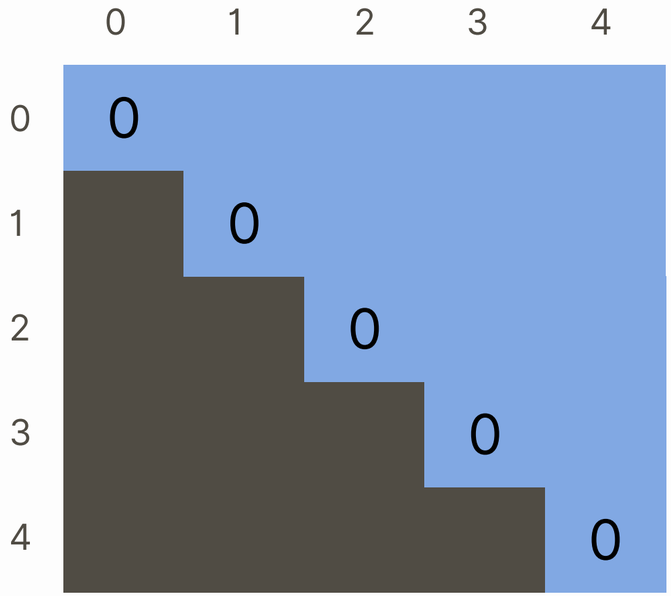
- base case → string length is 1, answer is 0
- string length 2, if both character is the same then answer is 0, answer is 1 otherwise
- for s[i...j]
  - if s[i] == s[j] then answer is minimum number of operations for string inside
  - otherwise, answer = 1 + minimum of:
    - operations needed for s[i+1..j] (remove or insert at left)
    - operations needed for s[i..j−1] (remove or insert at right)
  -

MAKING PALINDROME

What we know so far?

- base case → string length is 1, answer is 0
- string length 2, if both character is the same then answer is 0, answer is 1 otherwise
- for s[i...j]
  - if s[i] == s[j] then answer is minimum number of operations for string inside
  - otherwise, answer = 1 + minimum of:
    - operations needed for s[i+1..j] (remove or insert at left)
    - operations needed for s[i..j−1] (remove or insert at right)
  -

S = “abcea”

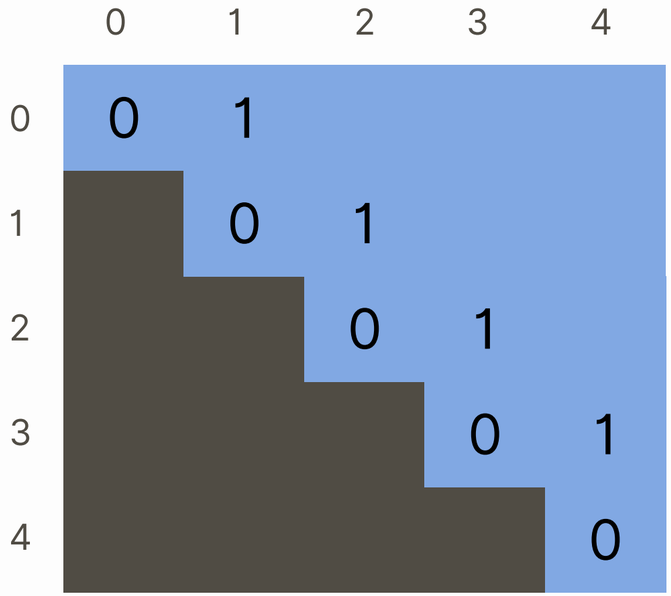


MAKING PALINDROME

What we know so far?

- base case → string length is 1, answer is 0
- string length 2, if both character is the same then answer is 0, answer is 1 otherwise
- for s[i...j]
  - if s[i] == s[j] then answer is minimum number of operations for string inside
  - otherwise, answer = 1 + minimum of:
    - operations needed for s[i+1..j] (remove or insert at left)
    - operations needed for s[i..j−1] (remove or insert at right)
  -

S = “abcea”

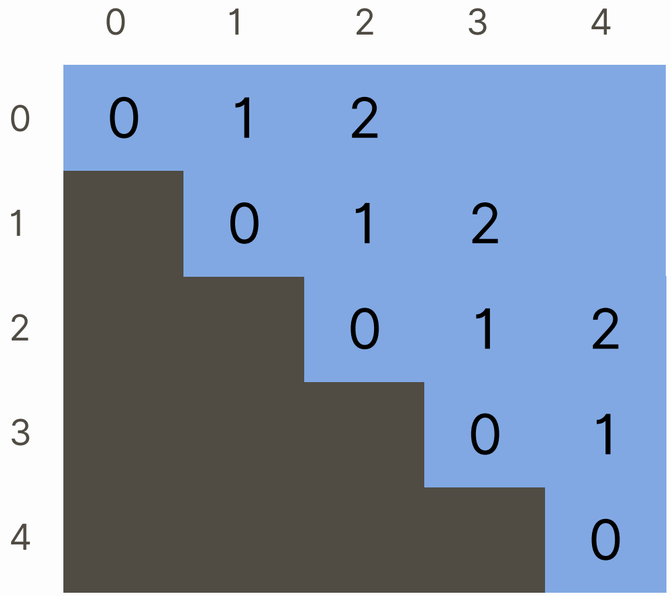


MAKING PALINDROME

What we know so far?

- base case → string length is 1, answer is 0
- string length 2, if both character is the same then answer is 0, answer is 1 otherwise
- for s[i...j]
  - if s[i] == s[j] then answer is minimum number of operations for string inside
  - otherwise, answer = 1 + minimum of:
    - operations needed for s[i+1..j] (remove or insert at left)
    - operations needed for s[i..j−1] (remove or insert at right)
  -

S = “abcea”

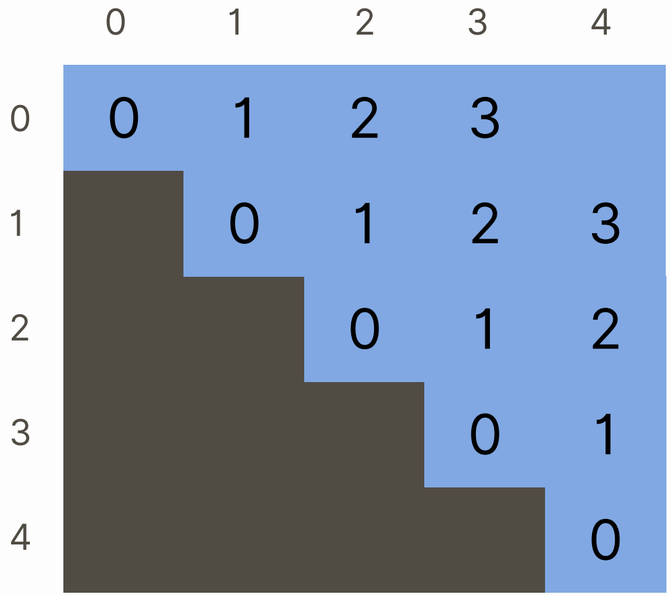


MAKING PALINDROME

What we know so far?

- base case → string length is 1, answer is 0
- string length 2, if both character is the same then answer is 0, answer is 1 otherwise
- for s[i...j]
  - if s[i] == s[j] then answer is minimum number of operations for string inside
  - otherwise, answer = 1 + minimum of:
    - operations needed for s[i+1..j] (remove or insert at left)
    - operations needed for s[i..j−1] (remove or insert at right)
  -

S = “abcea”



MAKING PALINDROME

What we know so far?

- base case → string length is 1, answer is 0
- string length 2, if both character is the same then answer is 0, answer is 1 otherwise
- for s[i...j]
  - if s[i] == s[j] then answer is minimum number of operations for string inside
  - otherwise, answer = 1 + minimum of:
    - operations needed for s[i+1..j] (remove or insert at left)
    - operations needed for s[i..j−1] (remove or insert at right)
  -

S = “abcea”

	0	1	2	3	4
0	0	1	2	3	2
1		0	1	2	3
2			0	1	2
3				0	1
4					0

MAKING PALINDROME

What we know so far?

- base case → string length is 1, answer is 0
- string length 2, if both character is the same then answer is 0, answer is 1 otherwise
- for s[i...j]
  - if s[i] == s[j] then answer is minimum number of operations for string inside
  - otherwise, answer = 1 + minimum of:
    - operations needed for s[i+1..j] (remove or insert at left)
    - operations needed for s[i..j−1] (remove or insert at right)
  -

S = “abcea”

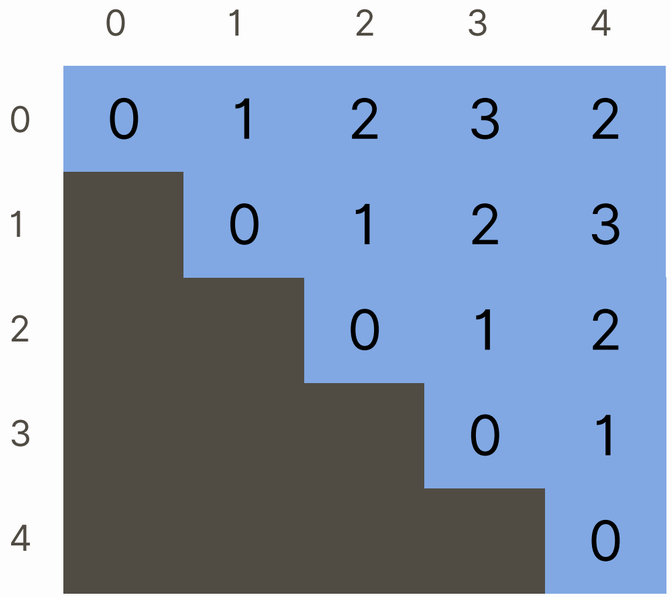
	0	1	2	3	4
0	0	1	2	3	2
1		0	1	2	3
2			0	1	2
3				0	1
4					0

MAKING PALINDROME

```
string s="abcea";
 int l = s.size();
 vector<vector<int>> dp(l, vector<int>(l, 0));

 for(int d=1;d<l;d++)
 {
  for(int i=0;i<l-d;i++)
  {
   int j = i+d;
   if(s[i]==s[j]) dp[i][j] = dp[i+1][j-1];
   else dp[i][j] = 1+min(dp[i+1][j], dp[i][j-1]);
  }
 }
 cout << dp[0][l-1];
```

S = “abcea”



PROBLEM TIME!

ICPC PAST PROBLEMS

- A Potion Shopping On This Wonderful World!
- Kid Rally

SELECTION ROUND PROBLEMS

- Christmas Gift
- Divisors

OTHER PROBLEMS

- Segi Tiga (TOI11)
- Carte (TOI16)